

```
fmt.Println("Best K:", bestK, "with Accuracy:", bestAccuracy)
```

Use feature selection for faster training: Unimportant features can slow down training. Use feature selection to improve efficiency.

```
import "github.com/sjwhitworth/golearn/filters"

// Select only the most important features
selector := filters.NewSelectBestFeatures(trainingData,
"Feature1", "Feature2")
reducedData := selector.Transform(trainingData)
```

Real-world applications of machine learning with GoLearn

GoLearn is gaining traction for its speed, efficiency, and scalability, making it ideal for real-world applications.
Finance: Classifies and detects fraud, predicts stock prices using regression.
Healthcare: Uses decision trees and logistic regression for accurate disease prediction.
E-commerce: Applies K-Means clustering for personalised product recommendations.

Comparison of GoLearn with other ML libraries

Machine learning is commonly associated with libraries like scikit-learn (Python), TensorFlow (Python/C++), and MLlib (Spark/Scala). However, GoLearn offers a unique blend of speed, efficiency, and simplicity.

Performance and efficiency: Performance is a key factor when choosing an ML library, as it directly impacts training time, inference speed, and scalability (see Table 1 for the comparison with other ML libraries).

GoLearn scores over other languages for the following reasons.

- **Compiled language:** Runs faster than Python-based ML libraries.

- **Lightweight:** Ideal for real-time applications and resource-constrained environments.
- **Efficient concurrency:** Leverages Go’s goroutines for parallel processing.
GoLearn should not be used:
- If you need deep learning (TensorFlow/PyTorch are better choices).
- If you work with Big Data (MLlib or Spark is more suitable).

Community support and documentation: Table 2 compares GoLearn with scikit-learn, TensorFlow and MLlib when it comes to community support and documentation.

Library	Community size	Documentation quality	Active development
GoLearn	Small	Moderate	Actively maintained
scikit-learn	Large	Excellent	Actively maintained
TensorFlow	Massive	Excellent	Constantly evolving
MLlib	Medium	Moderate	Slower updates

Table 2

Strengths of GoLearn: Go provides superior performance compared to Python-based ML libraries, while goroutines enable parallel execution without Python’s GIL bottleneck. Minimal dependencies make Go ideal for microservices and embedded ML apps.

Challenges and limitations of GoLearn

While GoLearn is a powerful and efficient machine learning library, there are several challenges and limitations that users should be aware of. These limitations are mostly related to the scope of features it offers, the community around it, and certain technical constraints.

Feature	GoLearn (Go)	scikit-learn (Python)	TensorFlow (Python/C++)	MLlib (Spark/Scala)
Execution speed	Fast (compiled)	Slower (interpreted)	Very fast (GPU/TPU support)	Optimised for distributed computing
Concurrency	Excellent (Goroutines)	Limited (GIL bottleneck)	Parallel processing	Highly scalable
Memory usage	Efficient	High memory overhead	Optimised	Distributed memory
Ease of use	Simple API	Simple API	Steep learning curve	Complex setup
Scalability	Good for medium datasets	Limited to single node	Large-scale ML	Best for Big Data

Table 1: A comparison of GoLearn with scikit-learn, TensorFlow, and MLlib